

HPC & Parallel Programming

GPU Programming

Kun Suo

Computer Science, Kennesaw State University

<https://kevinsuo.github.io/>

Outline

- GPU Motivation & Background
- GPU Architecture Fundamentals
- GPU Programming Model



Outline

- GPU Motivation & Background
- GPU Architecture Fundamentals
- GPU Programming Model

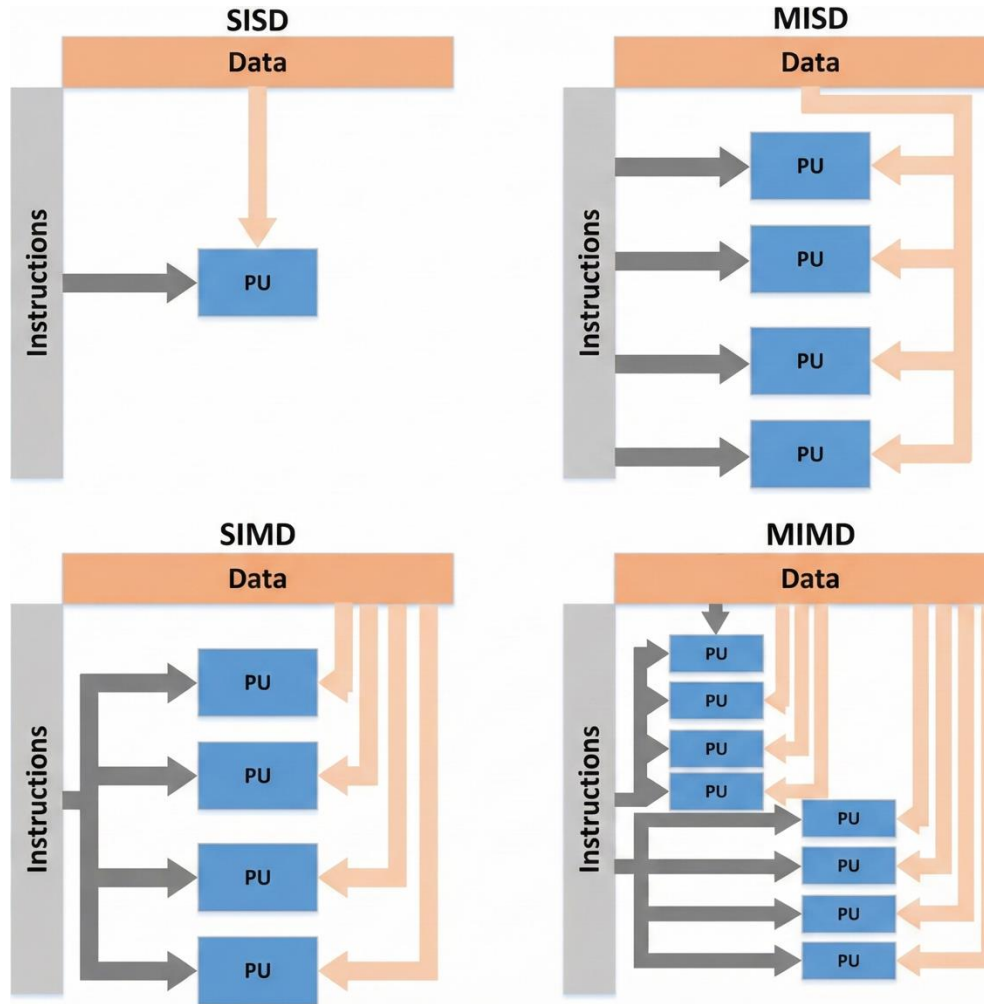


SIMD architecture

- SISD - Single instruction stream, single data stream
- SIMD - Single instruction stream, multiple data streams
 - New: SIMT – Single Instruction Multiple Threads (for GPUs)
- MISD - Multiple instruction streams, single data stream
 - No commercial implementation
- MIMD - Multiple instruction streams, multiple data streams
 - Tightly-coupled MIMD (shared memory, NUMAs)
 - Loosely-coupled MIMD (distributed memory system, e.g. cluster)



SIMD architecture



GPU

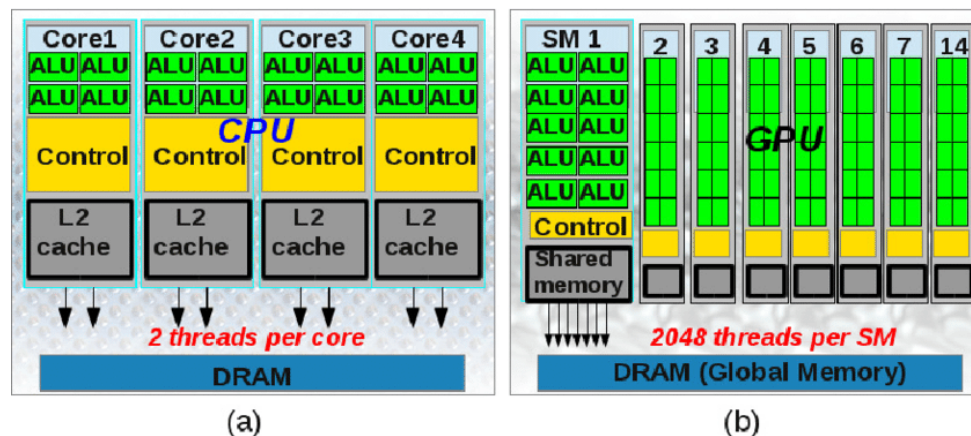
- A GPU (Graphics Processing Unit) is a computing device specifically designed to process images and videos.
 - Initially, GPUs were primarily used for rendering high-quality images and videos because they could process large amounts of pixel and vertex data in parallel.
 - With the increasing demand for large-scale parallel computing in science and engineering, GPUs have begun to be used in a wider range of applications, including scientific computing, machine learning, and data analysis.
- Compared to a CPU (Central Processing Unit), a GPU has many more cores (typically hundreds to thousands), allowing it to process a large number of computational tasks simultaneously.
 - Making GPUs highly efficient for handling large-scale parallel computing tasks, such as the training process in machine learning and deep learning.



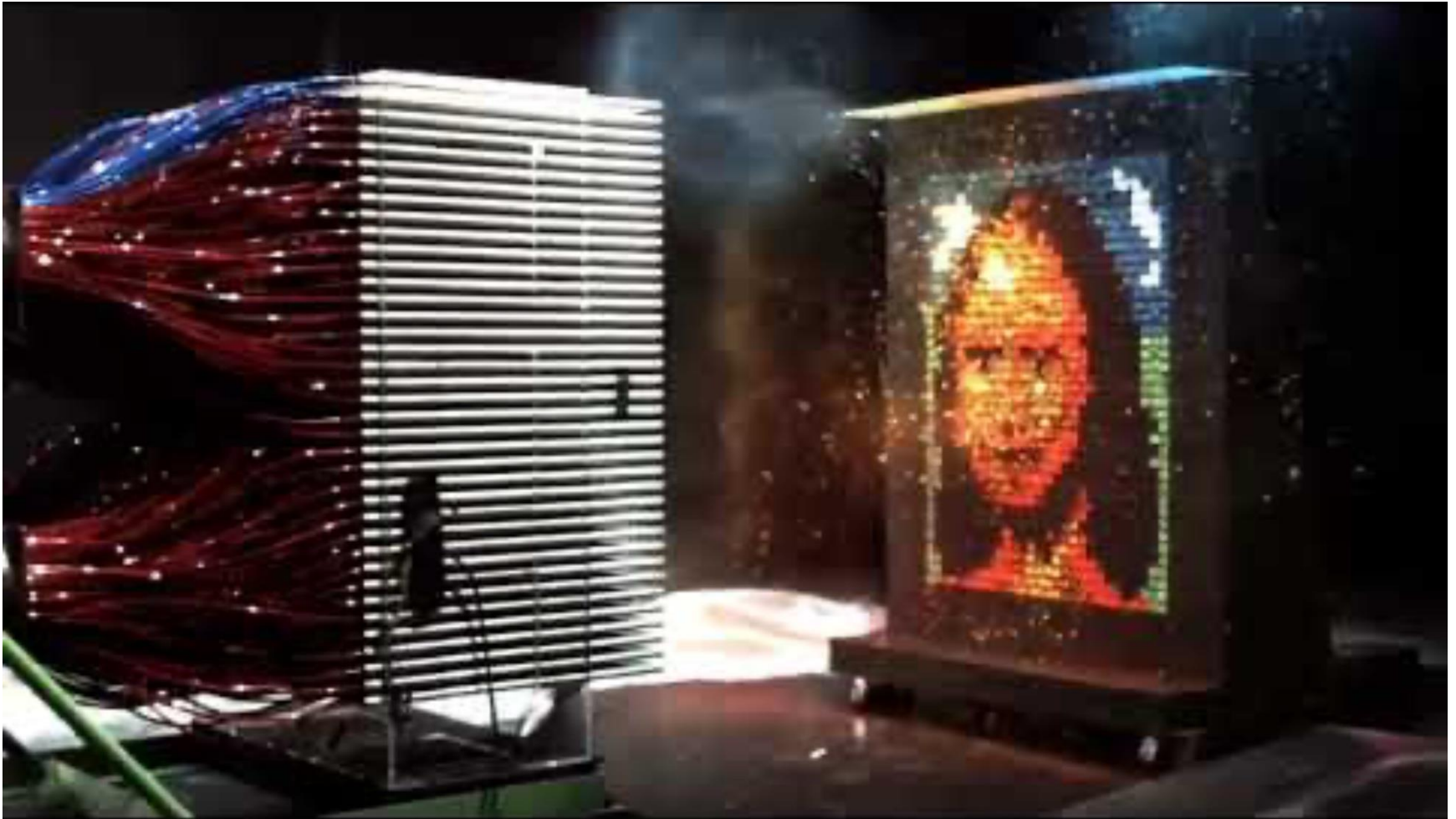
GPU vs CPU Philosophy

CPU	GPU
Few powerful cores	Thousands of simple cores
Latency optimized	Throughput optimized
Complex control logic	Massive parallel execution

GPUs sacrifice single-thread performance for throughput.

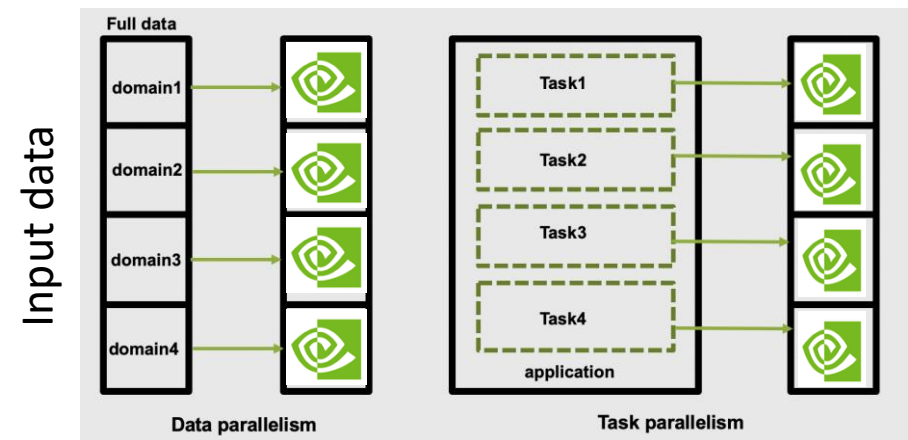


GPU vs. CPU



GPU Parallelism

- Massive parallelism:
 - Modern GPUs may contain **10,000+ compute cores**.
 - Example GPU: RTX 4090 ~ 16k cores.
- Parallel workload in GPUs:
 - Data parallel
 - Task parallel



Characteristics of GPUs

- **Massive core count:** GPUs contain thousands of cores, enabling simultaneous execution of many tasks.
- **High throughput:** While individual GPU cores are simpler than CPU cores, their scale delivers much higher overall processing throughput.
- **Parallel-first design:** GPUs are optimized for massive data parallelism, with each core handling independent, lightweight operations.
- **Dedicated memory:** GPUs use high-bandwidth device memory for fast access, though data transfer between CPU and GPU introduces overhead.



GPUs for accelerate parallel workloads

- **General-purpose computing:** Frameworks like **CUDA** and **OpenCL** enable GPUs to accelerate non-graphics workloads.
- **Big data analytics:** GPUs efficiently process large datasets for data science, financial modeling, and simulations.
- **Deep learning & AI:** GPUs accelerate neural network training and inference through massive parallel matrix operations.
- **Scientific simulation:** Widely used in molecular modeling, drug discovery, and climate simulation.
- **Graphics & multimedia:** Support 3D rendering, video encoding/decoding, and image processing.



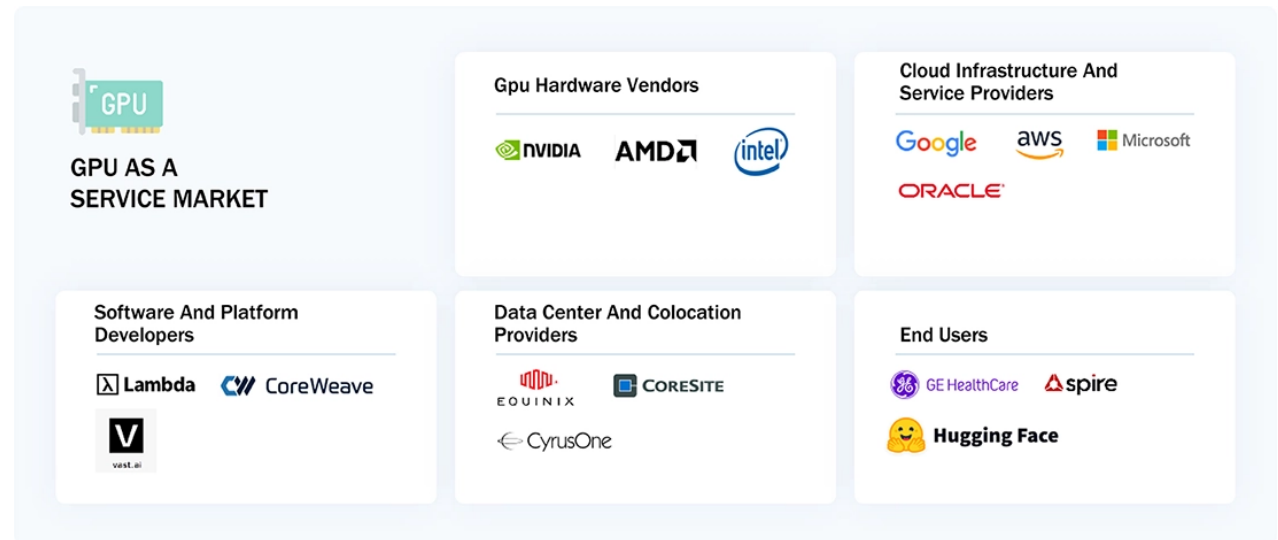
GPU Market & Ecosystem

- Major GPU vendors

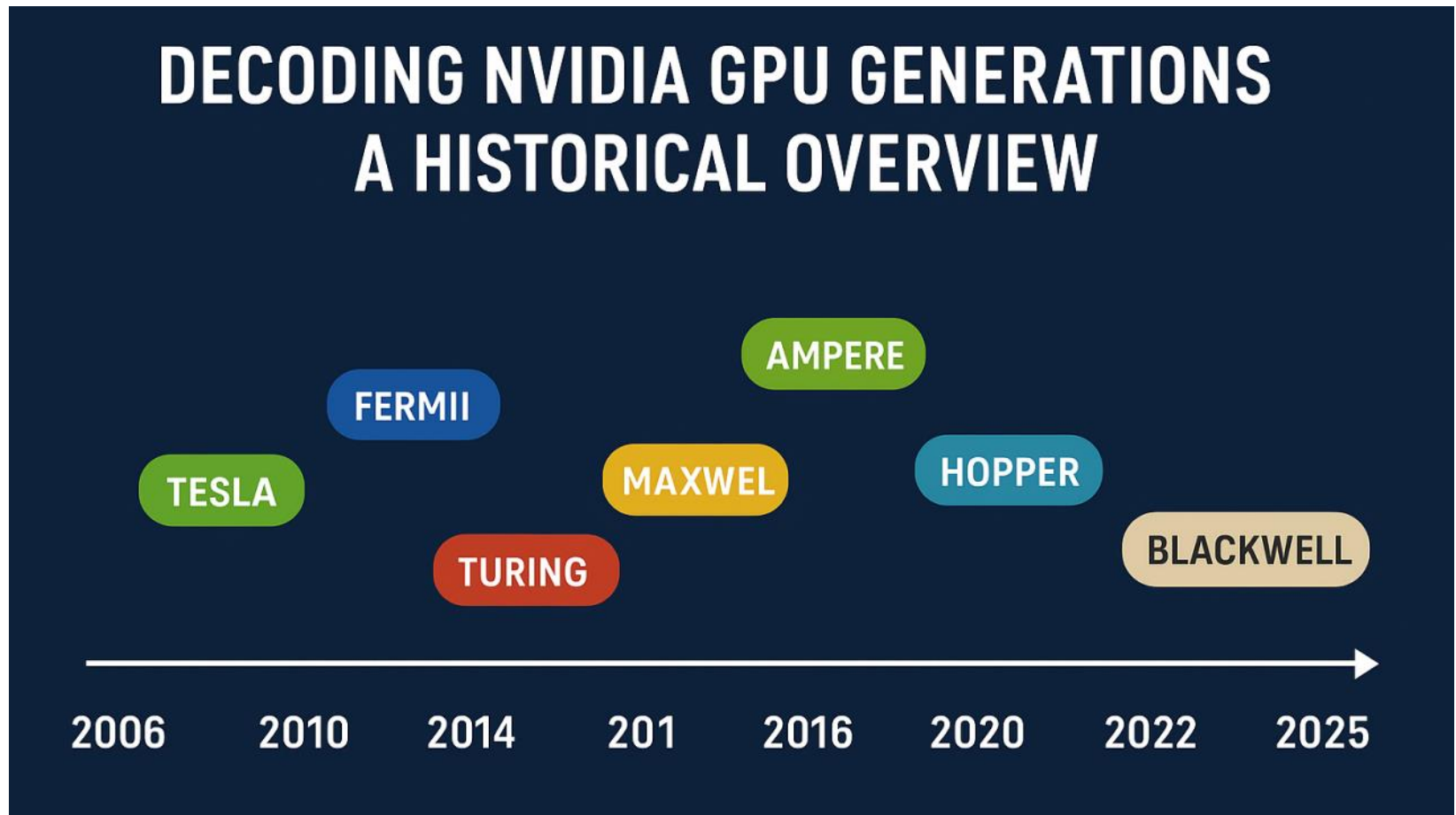
- NVIDIA
- AMD
- Intel
- Apple

- Platforms

- CUDA
- HIP
- OpenCL
- SYCL

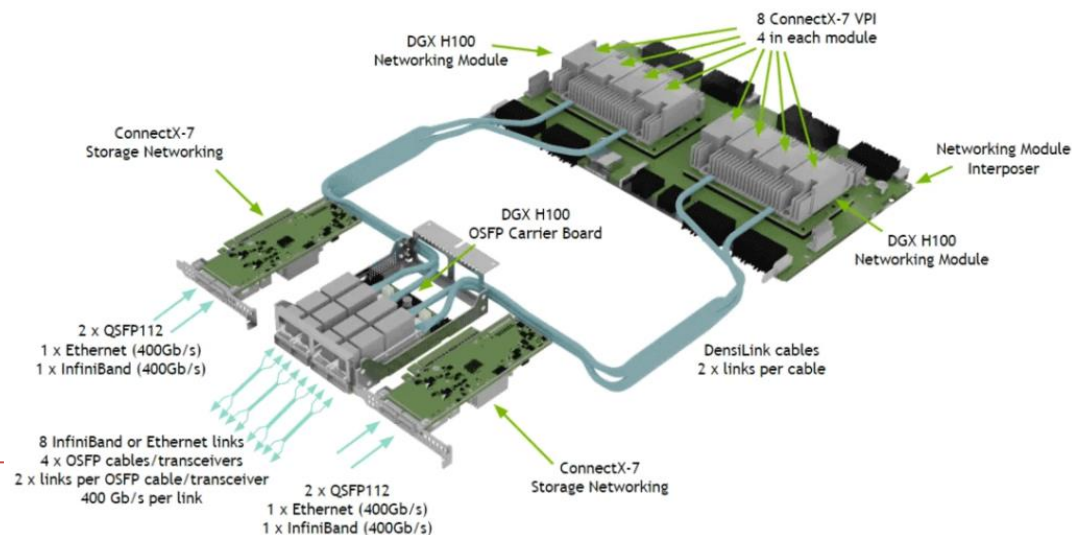


GPU Architecture Evolution



Multi-GPU Systems

- GPU interconnects
 - NVLink: A specialized, ultra-fast bridge for direct GPU-to-GPU communication that offers high bandwidth.
 - PCIe: The universal expansion interface used to connect the GPU to the motherboard and CPU for general data transfer.
 - InfiniBand: A high-speed networking standard designed for low-latency data exchange between separate server nodes in a cluster.



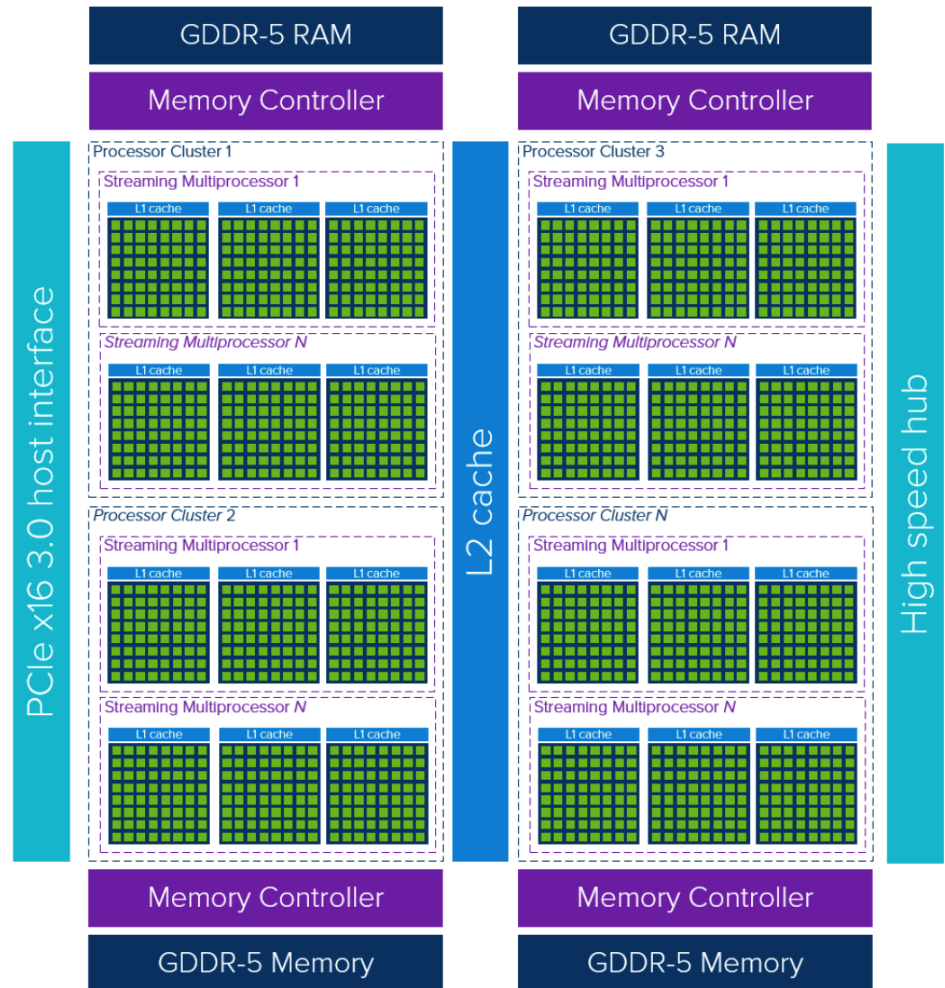
Outline

- GPU Motivation & Background
- GPU Architecture Fundamentals
- GPU Programming Model



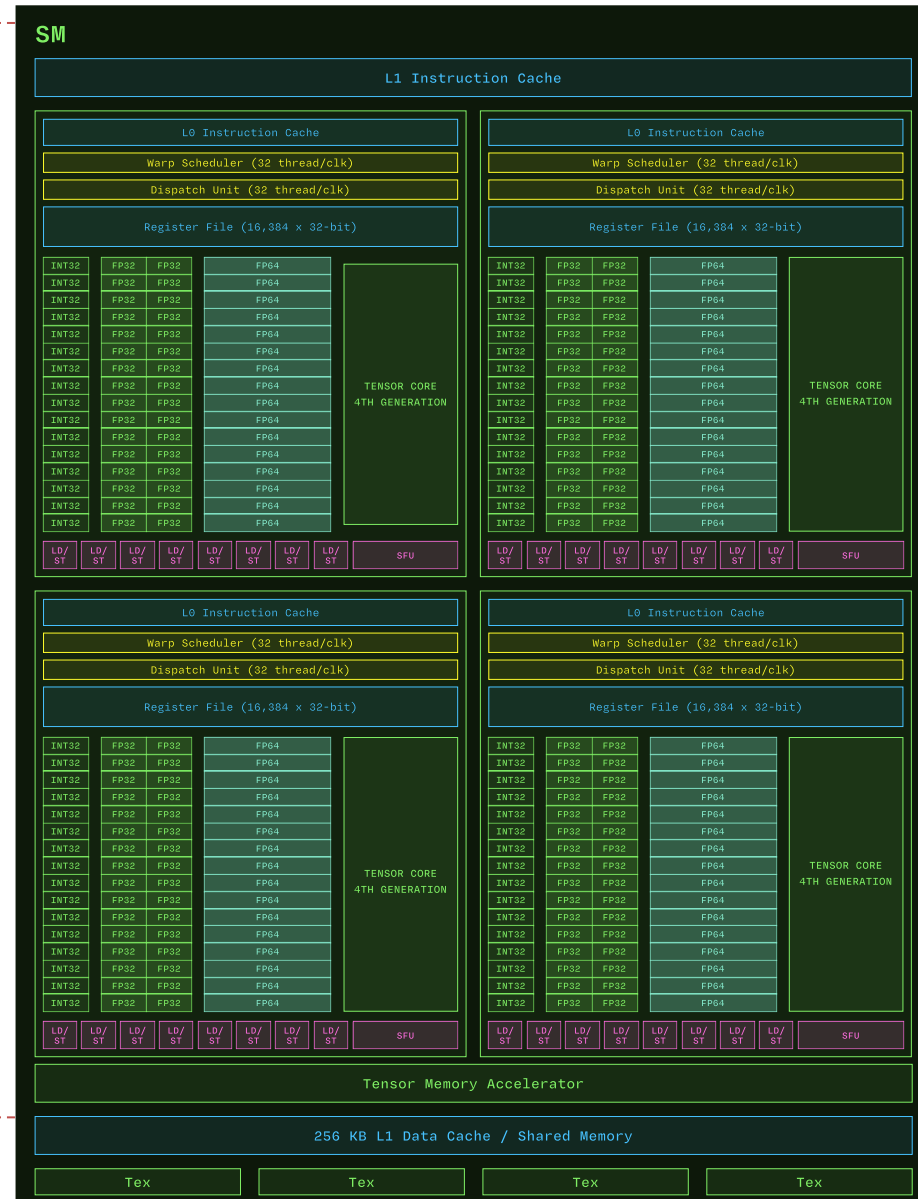
GPU Chip Layout and Architecture

- **Streaming Multiprocessors (SMs):** The primary engines that execute parallel instructions and mathematical logic.
- **Memory Controllers:** High-speed interfaces managing data flow between the GPU and external VRAM.
- **Cache Hierarchy:** Multi-level storage (L1/L2) that minimizes latency by keeping active data near the SMs.
- **Interconnect:** The internal high-bandwidth network linking SMs, caches, and memory controllers.



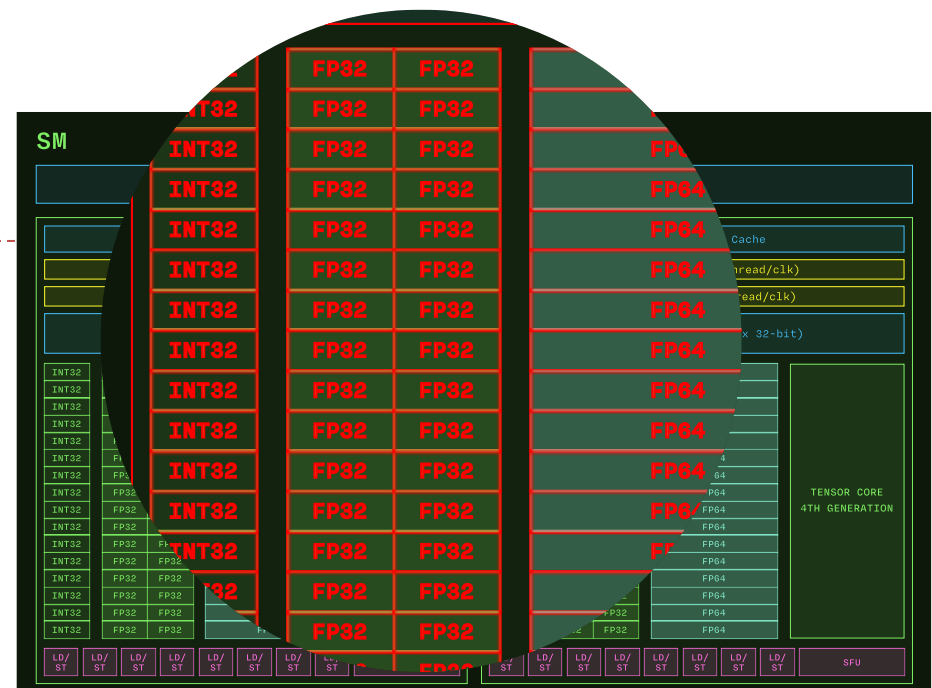
Streaming Multiprocessor (SM)

- SM = GPU equivalent of CPU core cluster
- SM contains:
 - CUDA cores
 - scalar arithmetic instructions
 - Tensor cores
 - operate on entire matrices with each instruction
 - RT cores
 - ray tracing
 - Registers
 - Shared memory

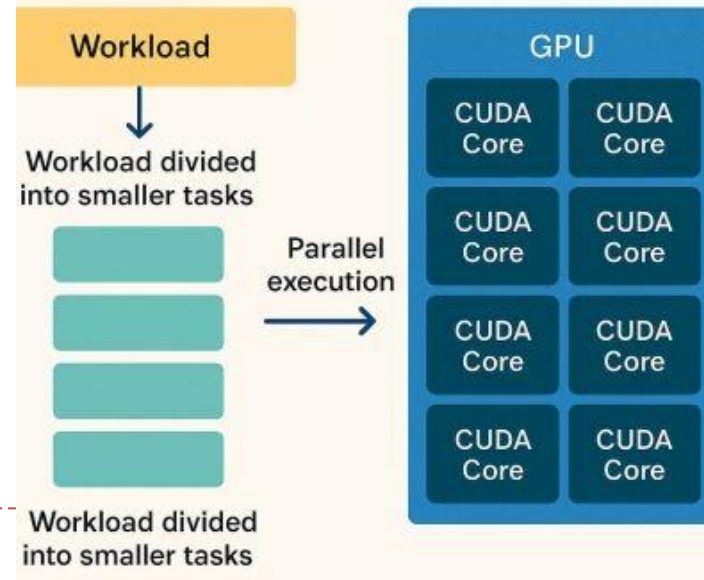


CUDA Cores

- CUDA cores execute:
 - FP32 arithmetic
 - integer operations
 - basic ALU instructions
- Each SM contains dozens to hundreds of CUDA cores.

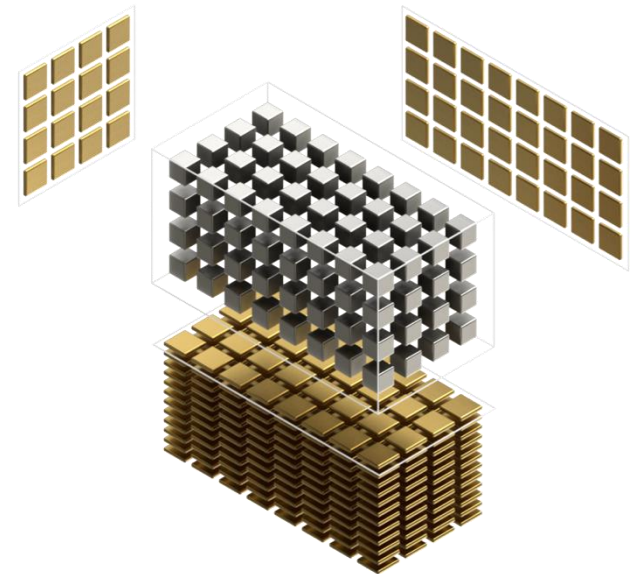
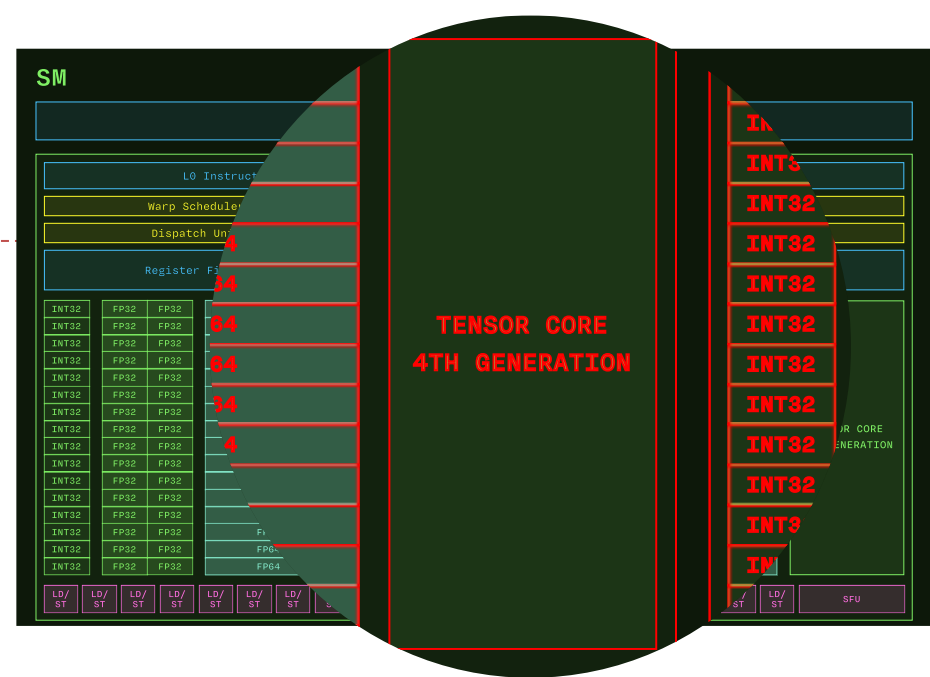


How CUDA Cores Work



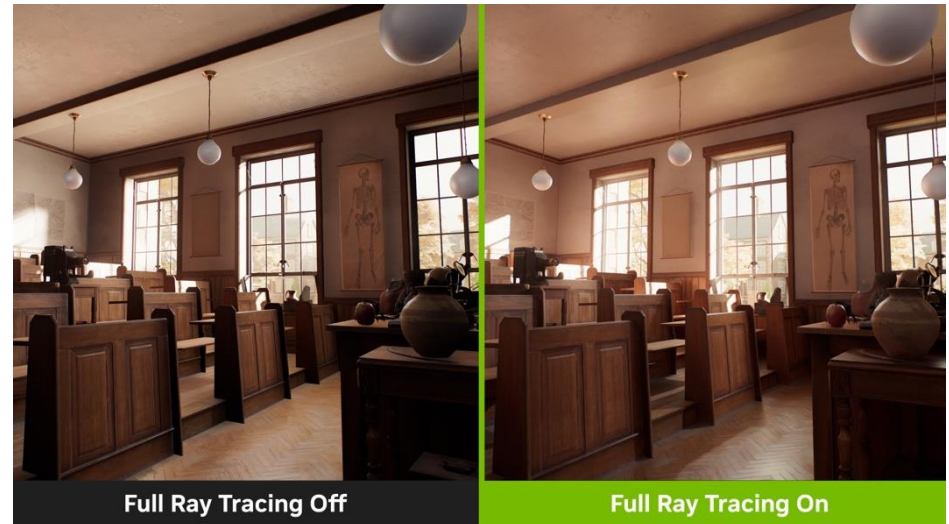
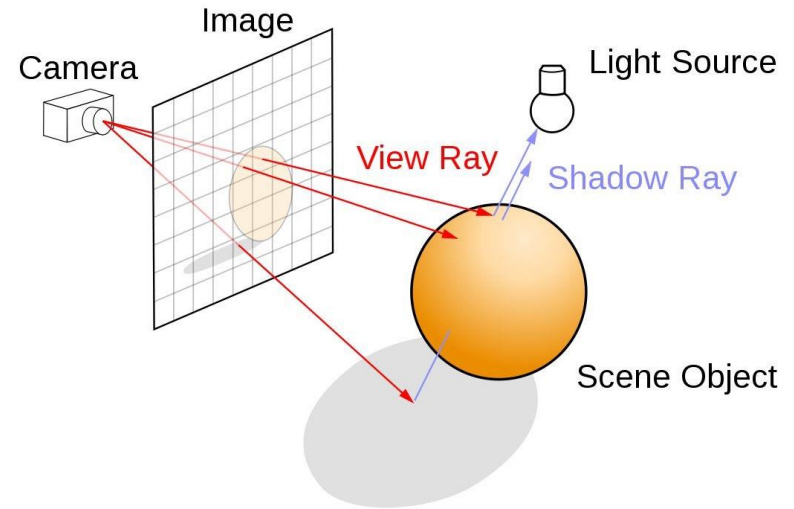
Tensor Cores

- Tensor cores accelerate:
 - matrix multiply-accumulate
 - AI training and inference
- Tensor cores perform operations like:
 - $D = A \times B + C$
- Example: H100 supports FP8 mixed precision.



Ray Tracing Cores

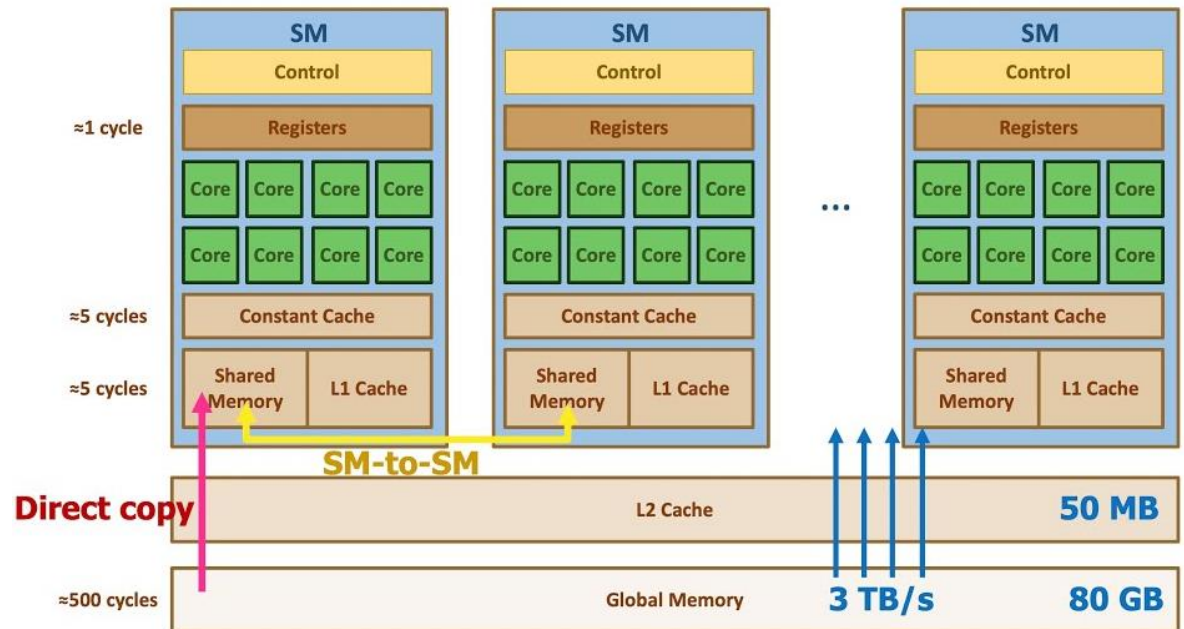
- RT cores accelerate:
 - ray-triangle intersection
 - bounding volume traversal
- Applications
 - real-time graphics
 - photorealistic rendering



GPU Memory Hierarchy

- Levels of GPU memory

- Registers
- Shared memory
- L1 cache
- L2 cache
- Global memory
- Host memory



Memory in Nvidia H100

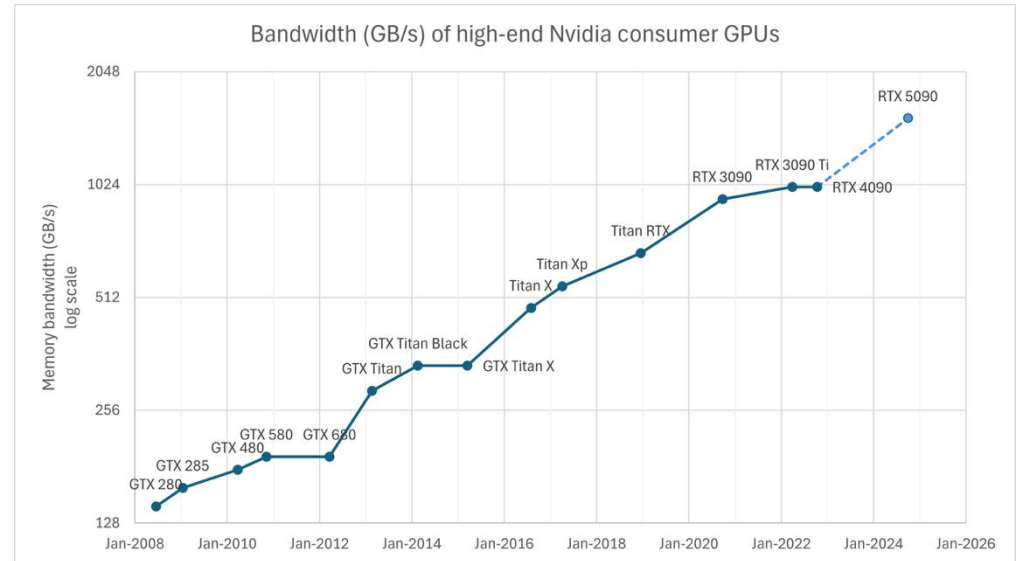
Memory Model

- **Constant and Texture Memory:** These are specific memory regions used for constant and texture data. They are optimized to provide higher read bandwidth
- **Local Memory:** Each thread has its own local memory.
- **Shared Memory:** Each block has a fast shared memory that threads within the block can use for communication
However, different blocks cannot share this memory
- **Global Memory:** This is a large block of DRAM that all threads can access, but access speed is relatively slow. Data transfer between the host (CPU) and the device (GPU) is typically done through global memory



Memory Bandwidth

- Typical GPU bandwidth:
 - ~1 TB/s memory bandwidth
- Why bandwidth matters:
 - feed thousands of cores
 - avoid memory bottlenecks



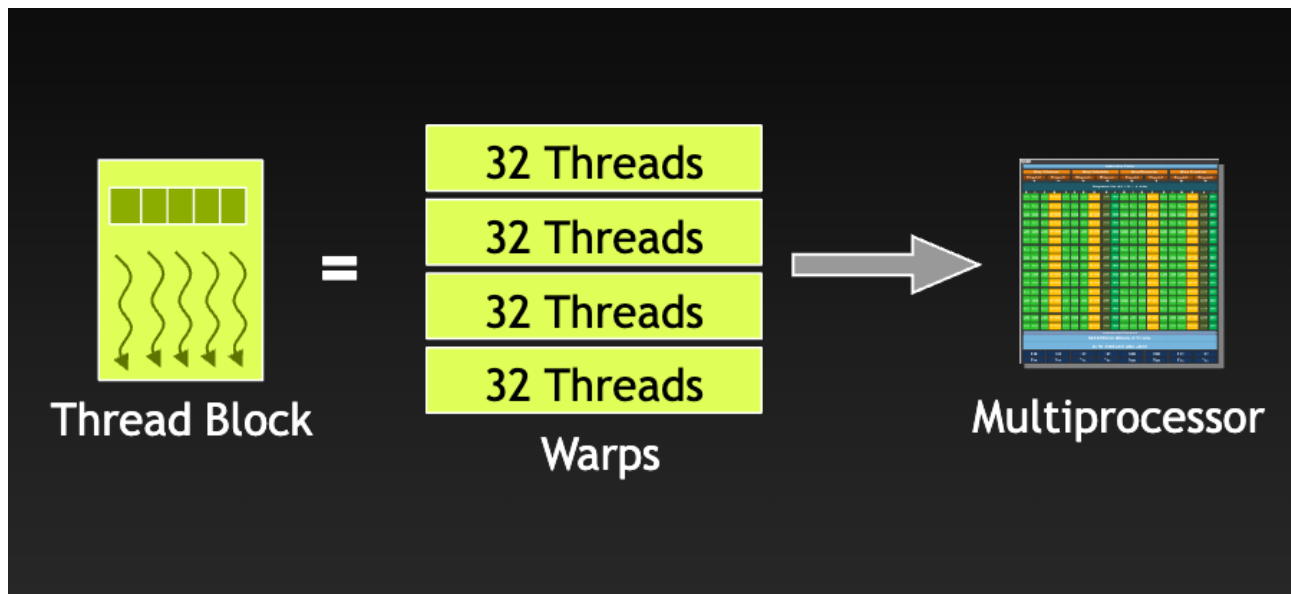
GPU Memory Bandwidth Comparison

Here's a summary of the memory bandwidth for each GPU:

GPU	Memory Bandwidth	VRAM
RTX A6000 (48GB)	768 GB/s TechPowerUp	48GB
RTX A5000 (24GB)	768 GB/s Pnypartners	24GB
RTX A4000 (16GB)	448 GB/s StorageReview	16GB
NVIDIA A40 (48GB)	696 GB/s CUDO Compute	48GB
NVIDIA A100 (40GB) PCIe	1.6 TB/s (1,600 GB/s) Wikipedia	40GB
NVIDIA A100 (80GB) PCIe	2.0 TB/s (2,000 GB/s) Datacrunch	80GB
NVIDIA A10 (24GB)	600 GB/s NVIDIA	24GB

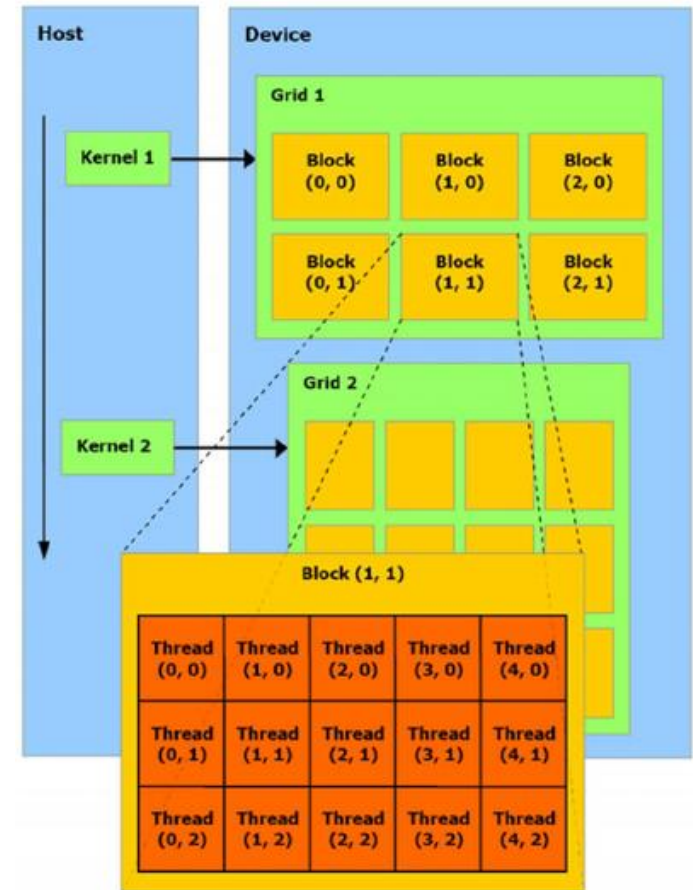
Warp Execution

- Warp: In NVIDIA's GPU architecture, threads do not execute individually, but rather in groups of 32 threads (called a **warp**) simultaneously.
- All threads within a warp execute the same instruction, but on different data. (SIMD)



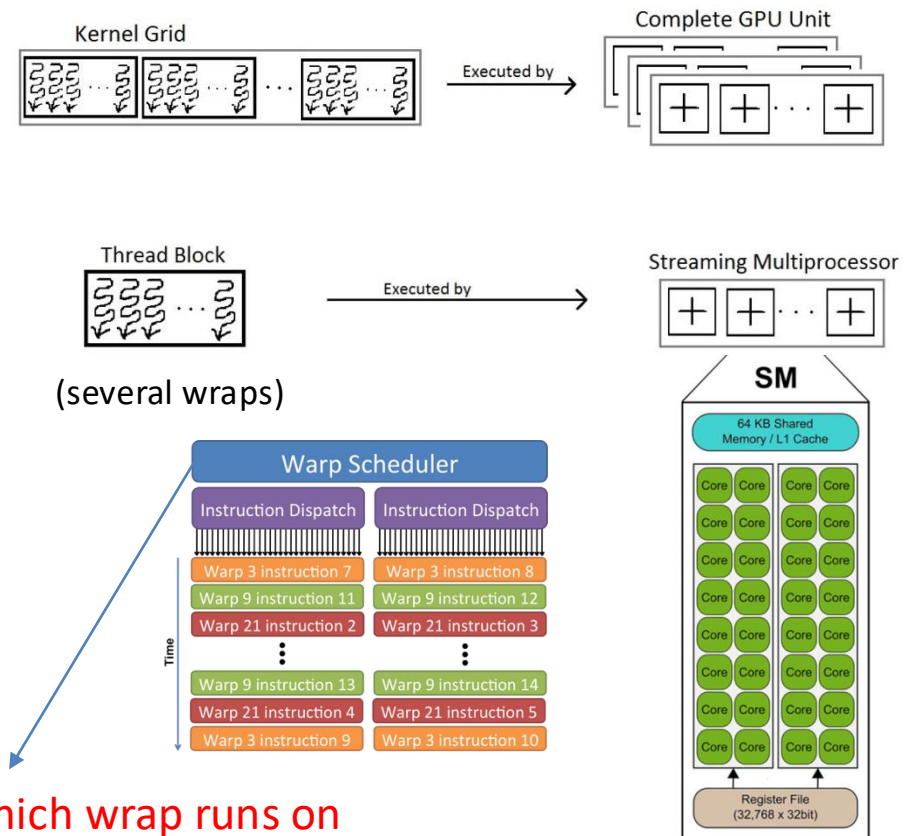
Thread Hierarchy

- **Kernel:** A function that runs on the GPU and can process multiple data elements in parallel. → the *job description*
- **Thread:** The basic unit of execution for a kernel. Each thread processes one or more data elements. → an *individual worker*
- **Block:** A collection of threads (several warps). All threads within a block can share certain memory resources. → a *team within the company*
- **Grid:** A collection of blocks. The entire grid is the set of all threads that execute the kernel in parallel on the GPU. → the *entire company working on the job*



Warp Scheduling (GPU Thread Scheduling)

- GPU employs a hierarchical architecture: at the highest level, the Grid is responsible for allocating tasks to specific Streaming Multiprocessors (SMs)
- Within each SM, warps are responsible for scheduling threads to execute the current tasks
- Once a thread block is launched on a SM, all of its warps remain resident on that SM until their execution is complete



Which warp runs on
which cores in SM

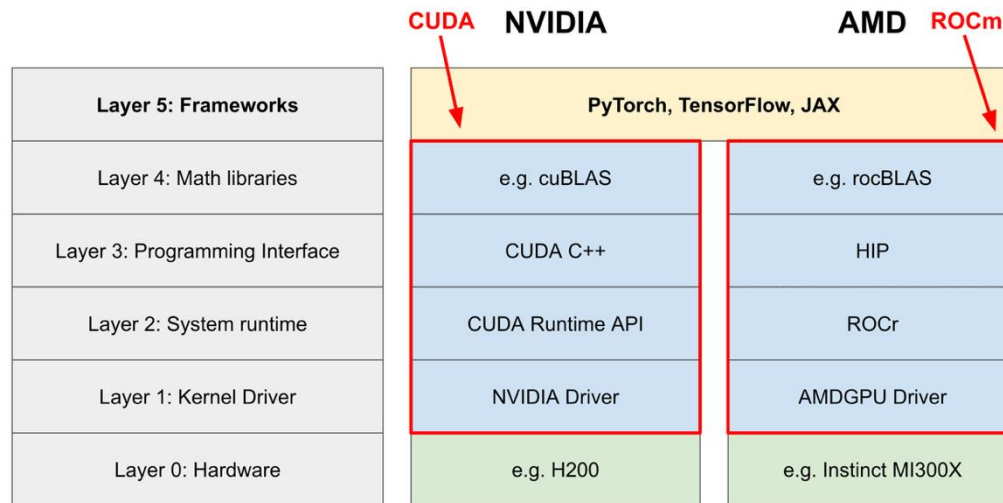
Outline

- GPU Motivation & Background
- GPU Architecture Fundamentals
- GPU Programming Model



GPU Programming Frameworks

- **CUDA:** a proprietary parallel computing platform and API model created by NVIDIA
- **OpenCL:** an open, royalty-free standard managed by the Khronos Group that enables cross-platform parallel programming across diverse hardware
- **SYCL:** a higher-level, C++ based abstraction layer built on top of OpenCL
- **HIP:** a C++ runtime API created by AMD



What is CUDA?

- CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA.
- It allows developers to directly use NVIDIA GPUs to perform general-purpose computing tasks, achieving higher performance than traditional CPU computing.
- CUDA essentially transforms the GPU from a purely graphics rendering device into a general-purpose parallel processor.



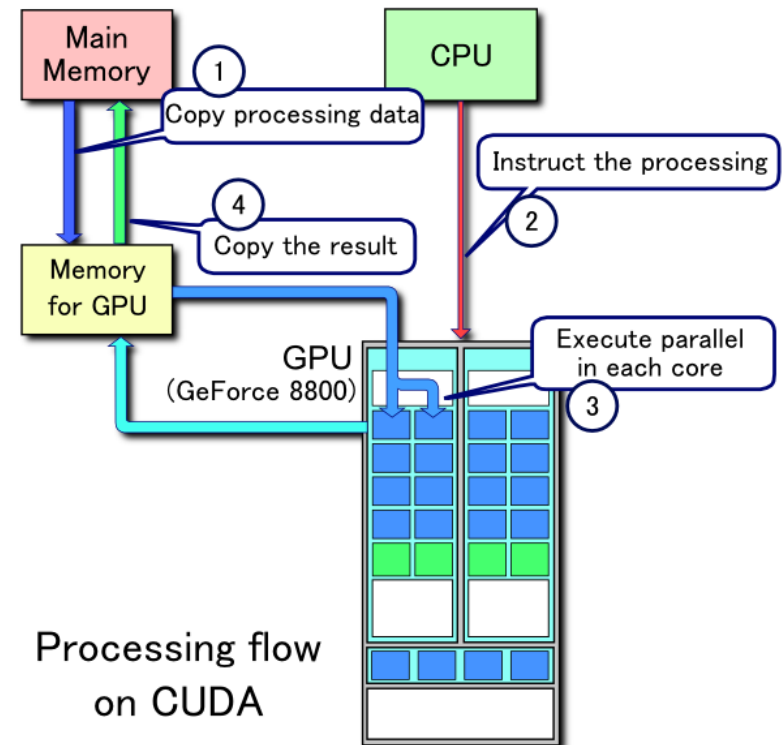
Key features of CUDA

- **Parallel processing:** NVIDIA GPUs consist of thousands of processing cores that can process a large number of tasks in parallel.
- **Suitable for highly parallel computing tasks:** such as graphics rendering, scientific computing, and deep learning.
- **Integration with C/C++ languages:** CUDA provides an extension to the C language, allowing developers to directly write functions (called kernels) that run on the GPU within their C/C++ code.



The basic steps of CUDA programming

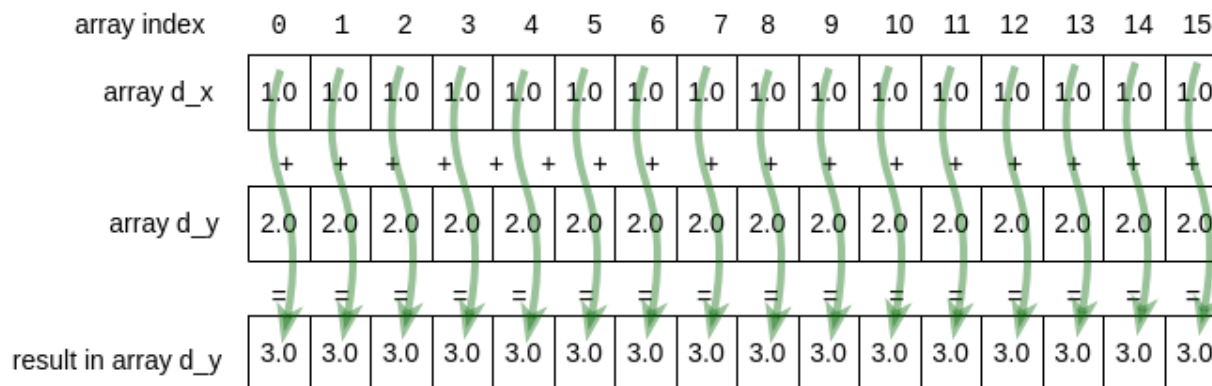
- **Data allocation:** Allocate and initialize memory between the host (CPU) and the device (GPU).
- **Data transfer:** Transfer data from the host to the device.
- **Kernel launch:** Specify the grid and block sizes and launch the kernel on the GPU.
- **Data retrieval:** Transfer the results from the device back to the host.
- **Cleanup:** Release memory resources on both the device and the host.



Example: Vector addition on CUDA

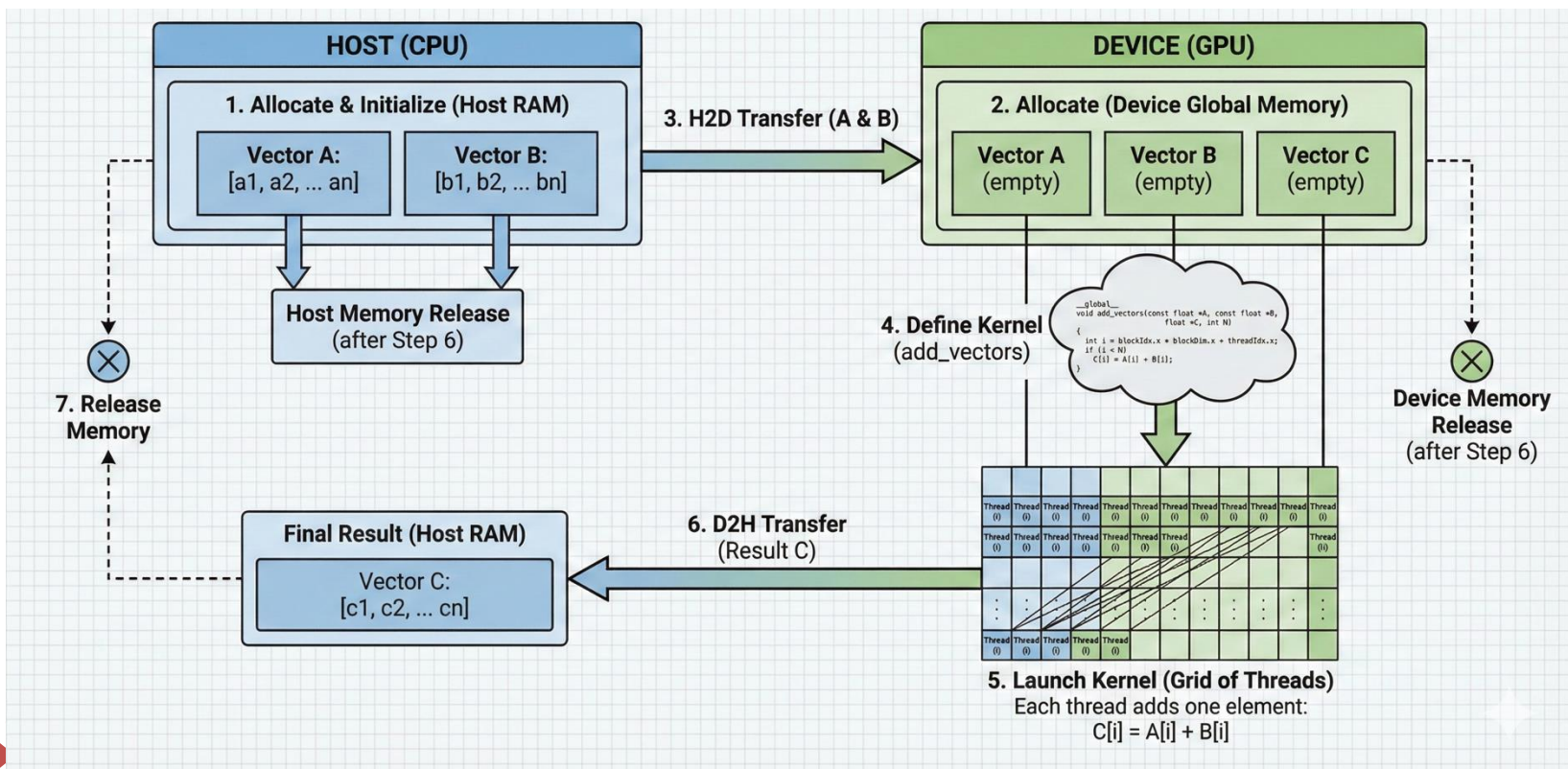
- Vector addition kernel $C[i] = A[i] + B[i]$

```
__global__ void vecAdd(float *A, float *B, float *C){  
    int i = threadIdx.x;  
    C[i] = A[i] + B[i];  
}
```



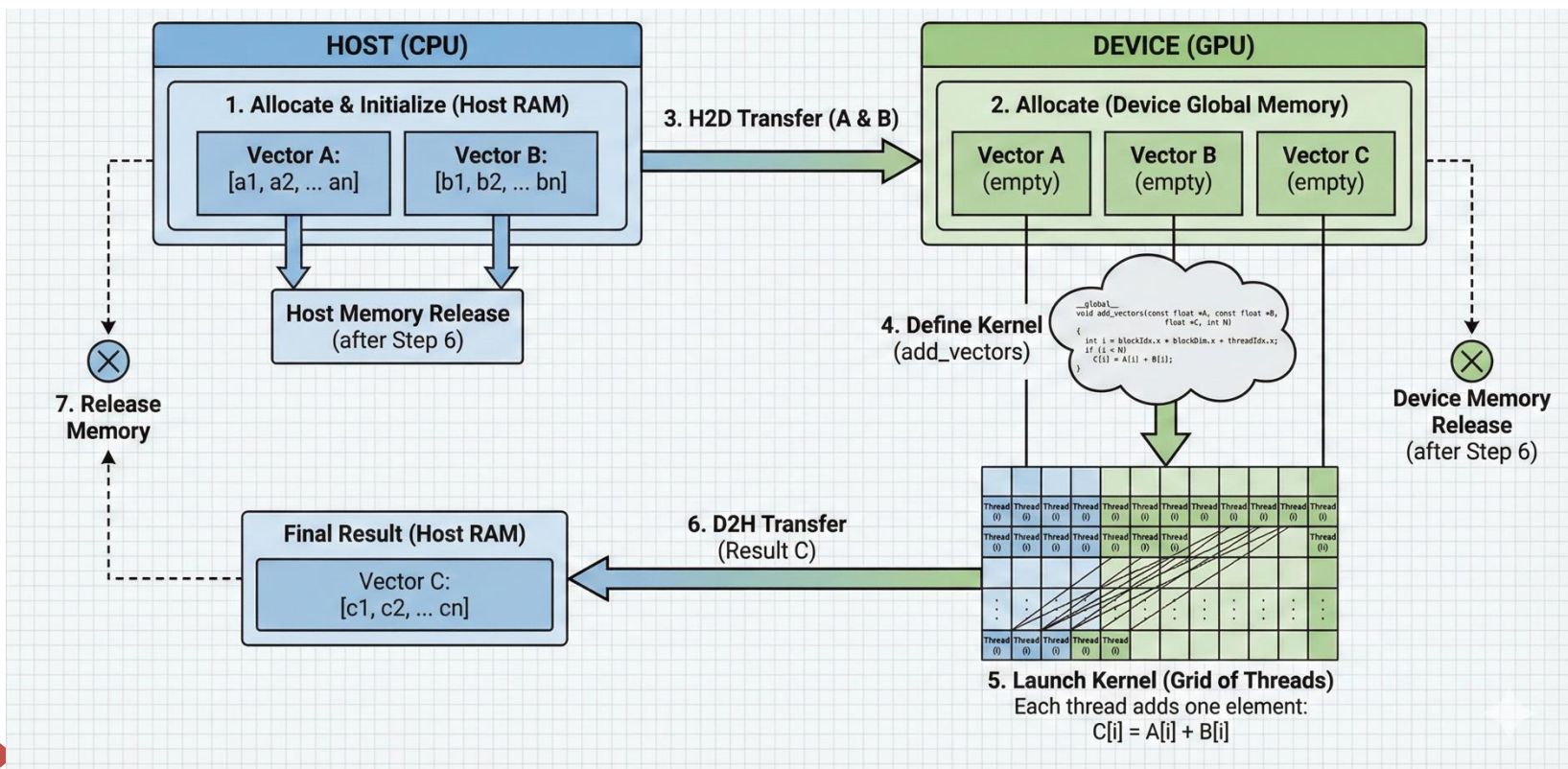
Example: Vector addition on CUDA $C[i] = A[i] + B[i]$

1. Allocate memory on host and initialize vectors A and B.
2. Allocate memory for vectors A, B, and C on the device.
3. Transfer vectors A and B from the host to the device.



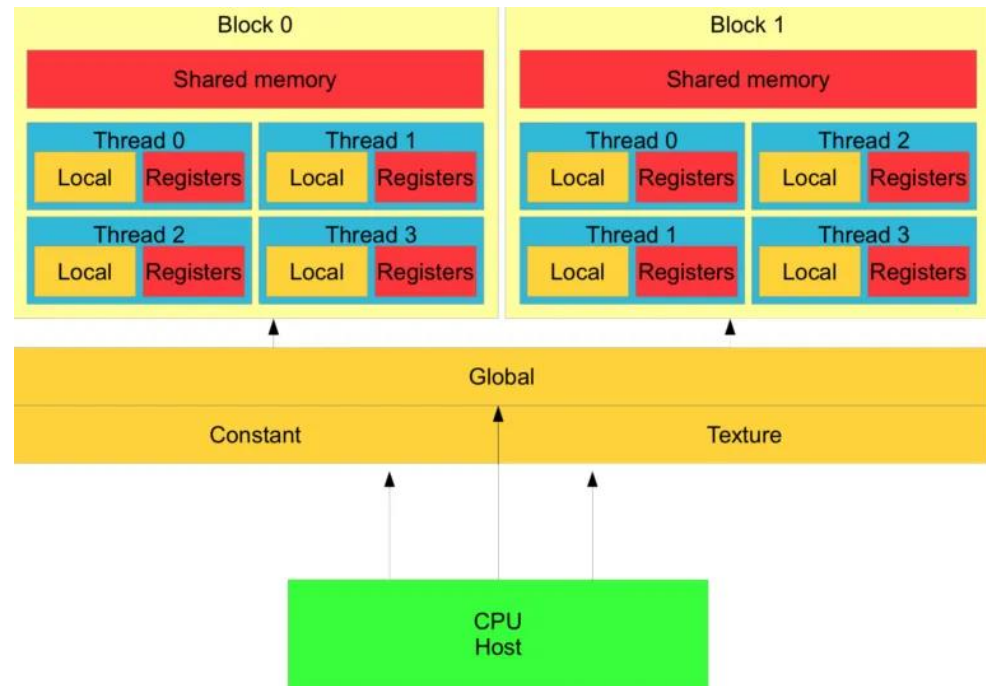
Example: Vector addition on CUDA $C[i] = A[i] + B[i]$

4. Define a kernel where each thread performs the addition of one element.
5. Launch the kernel, specifying enough threads to cover all vector elements.
6. Transfer the resulting vector C from the device back to the host.
7. Release all allocated memory.



Memory Types in CUDA

- Off-Chip Memories (High Latency, Large Capacity)
 - Global Memory
 - Constant Memory
 - Texture Memory
- On-Chip Memories (Low Latency, Small Capacity)
 - Shared Memory
 - Local Memory
 - Registers
- Caches and Other Components
 - Caches (L1/L2)

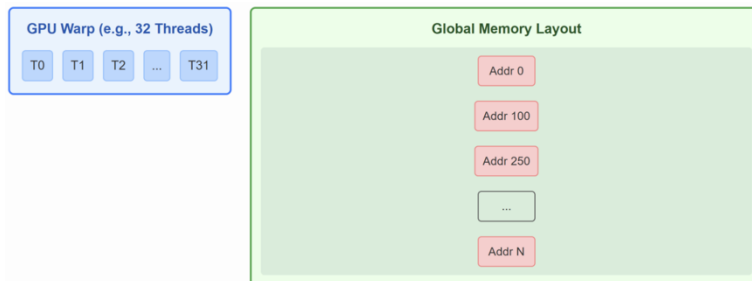


GPU Memory Coalescing

- Example: Think of a "warp" of GPU threads (typically 32 threads) as a team of workers. They all need to fetch supplies (data) from a warehouse (GPU global memory).

Scenario 1: The Uncoalesced Mess

Worker 1 goes to Aisle 1, Shelf A.
Worker 2 goes to Aisle 5, Shelf X.
Worker 3 goes to Aisle 10, Shelf Q.
...and so on for all 32 workers.



T0 → Addr 0 (Separate Transaction)
T1 → Addr 100 (Separate Transaction)
T2 → Addr 250 (Separate Transaction)
...
T31 → Addr N (Separate Transaction)
Many small, inefficient memory transactions.

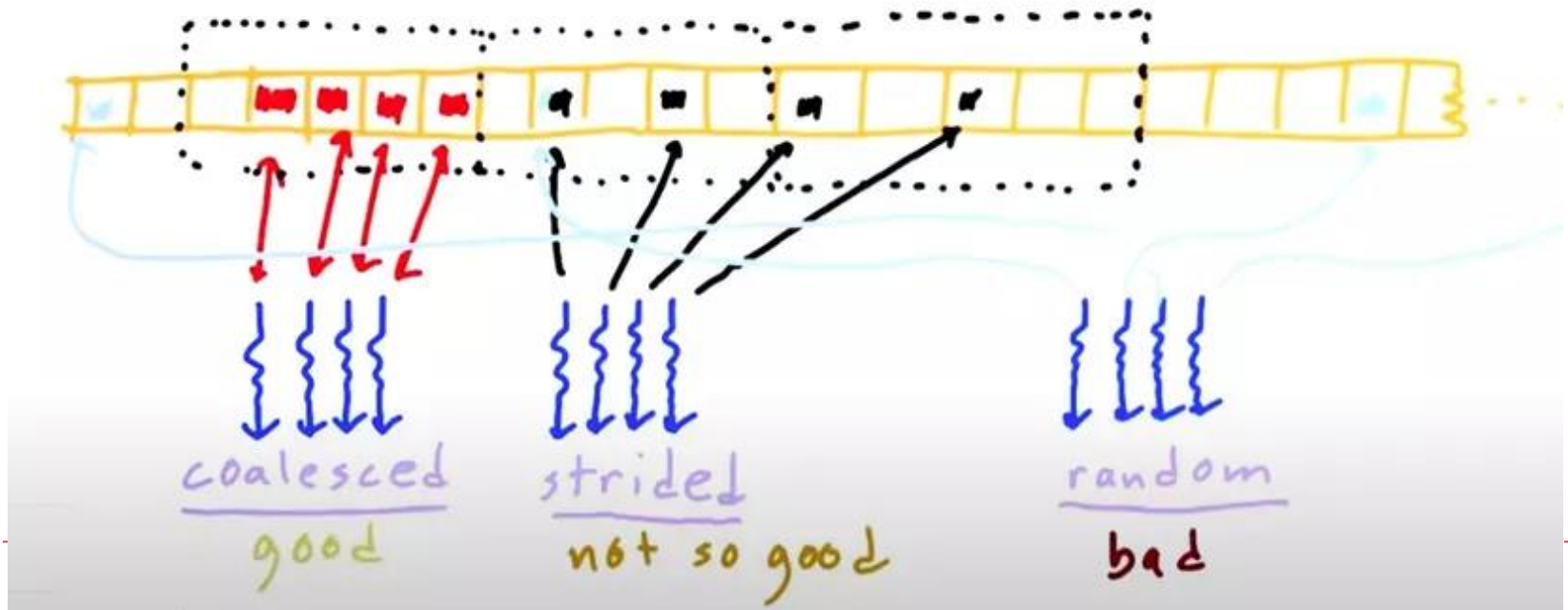
Scenario 2: The Coalesced Dream

Worker 1 needs item 1 from Aisle 1, Shelf A.
Worker 2 needs item 2 from Aisle 1, Shelf A.
Worker 3 needs item 3 from Aisle 1, Shelf A.
...and so on.



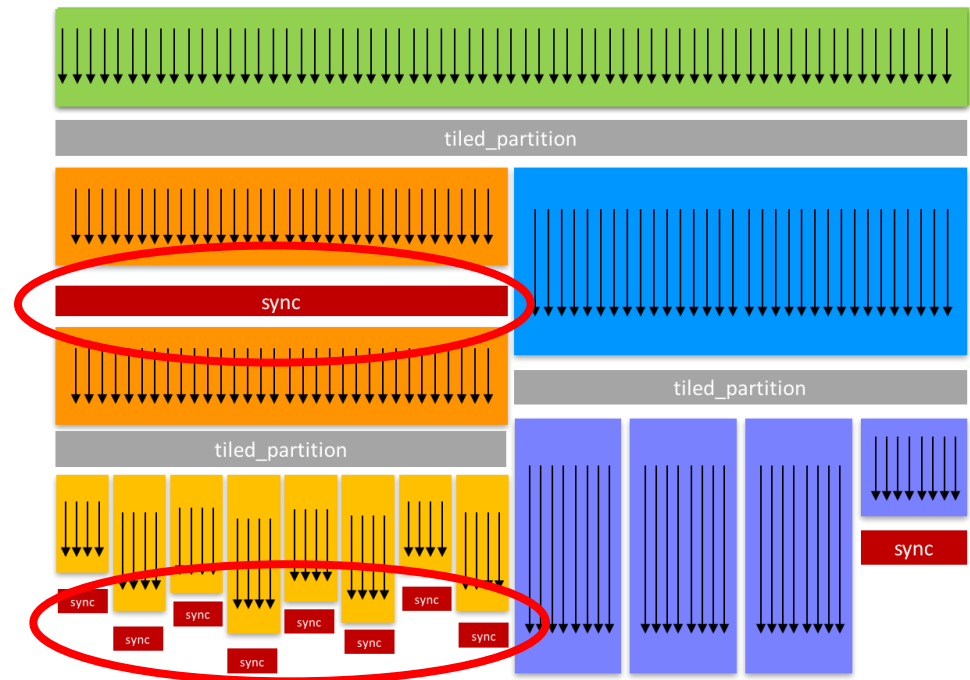
GPU Memory Coalescing

- Uncoalesced threads spend most of their time **waiting for data** instead of doing actual computation
 - Increased Latency: Individual, slow memory fetches.
 - Wasted Bandwidth: The memory bus isn't used to its full potential.
 - Stalled Cores: Your powerful GPU cores sit idle, data-starved.



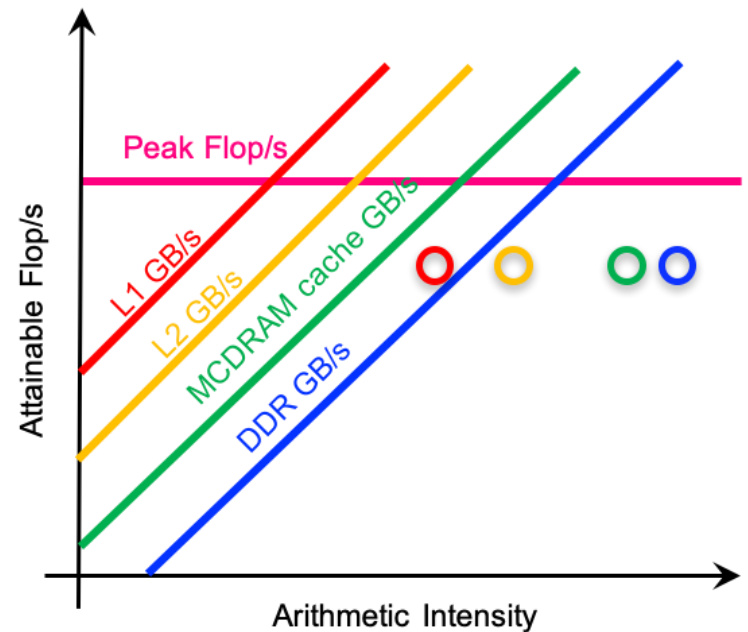
Synchronization

- GPU synchronization ensures that tasks running on the GPU and between the CPU and GPU execute in a correct and predictable order
- Thread synchronization primitives:
 - `__syncthreads()`
- Use cases:
 - Shared memory coordination
 - Reduction algorithms

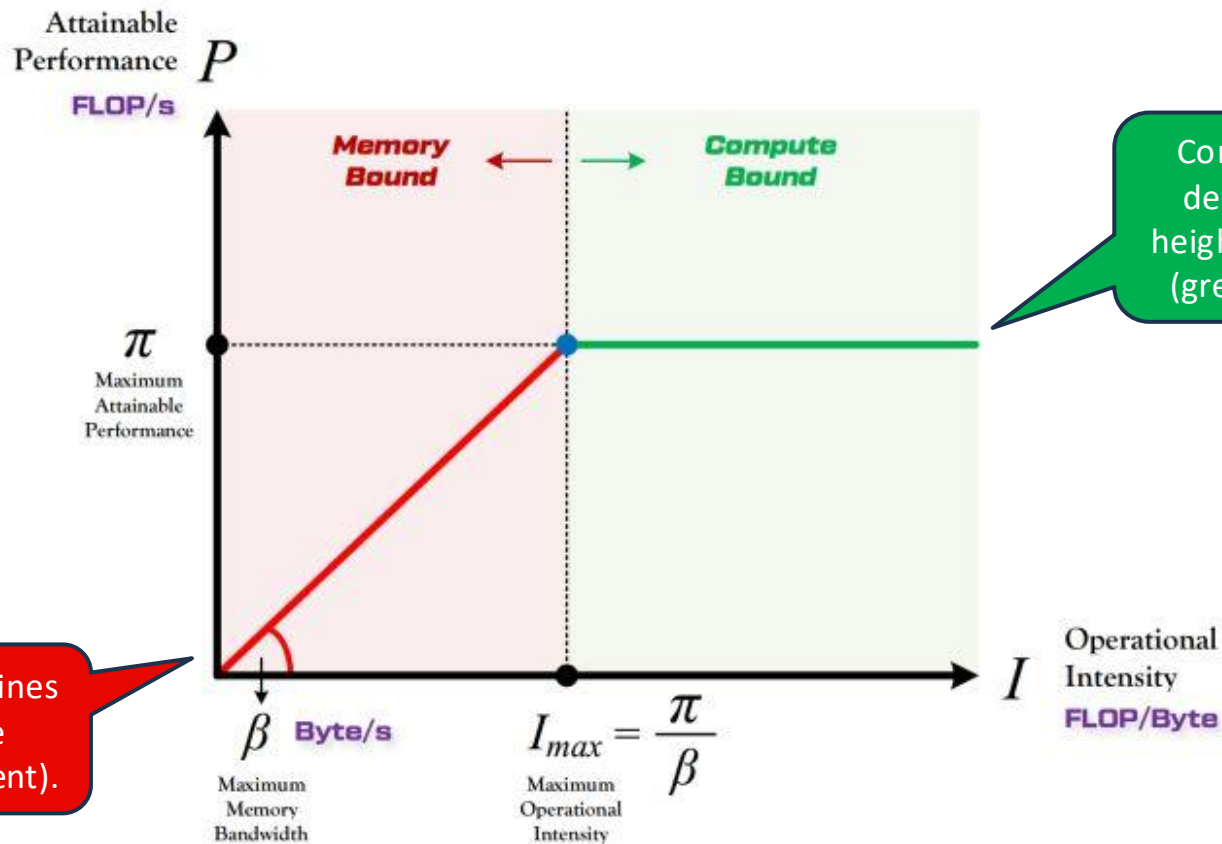


GPU Performance Model

- GPU performance limited by
 - memory bandwidth
 - compute throughput
 - latency hiding
- The Roofline model describes the theoretical upper bound of a program's performance under the constraints of computing power, bandwidth, etc. on a computing platform.



Roofline Model



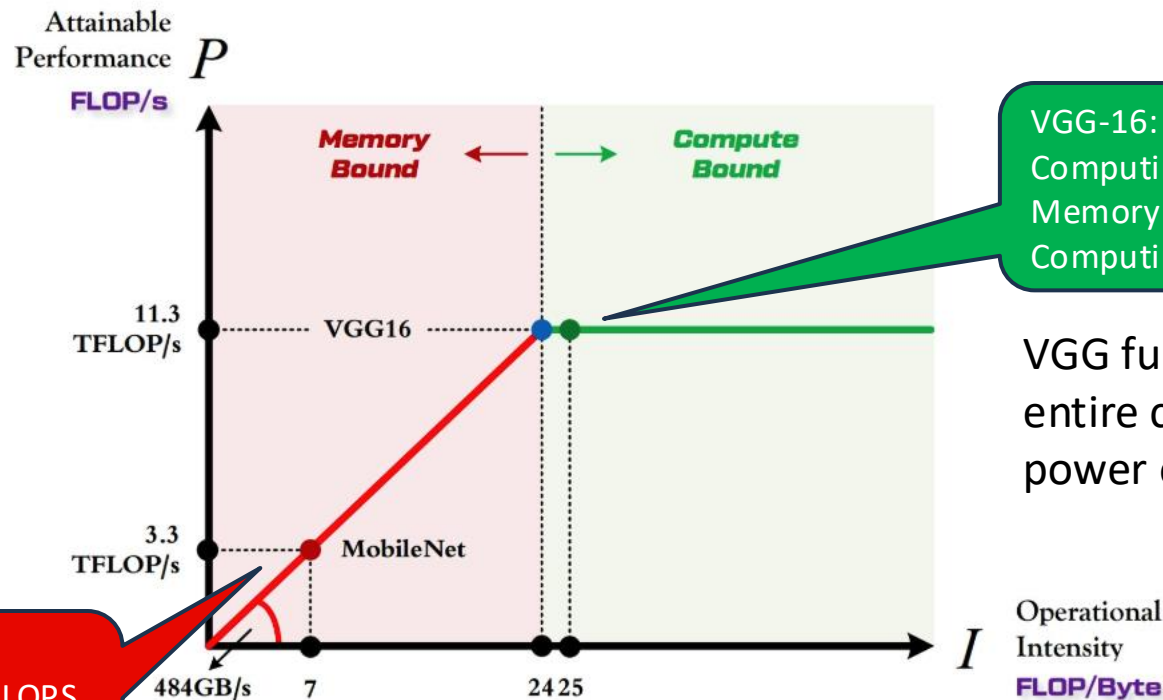
Roofline Model Example



The compute power of the 1080Ti is $P = 11.3$ TFLOPS/s

The bandwidth of the 1080Ti is $\beta = 484$ GB/s

Therefore, the maximum compute intensity of the 1080Ti computing platform is $I_{\max} = 11.3\text{T}/484\text{G} \approx 24$



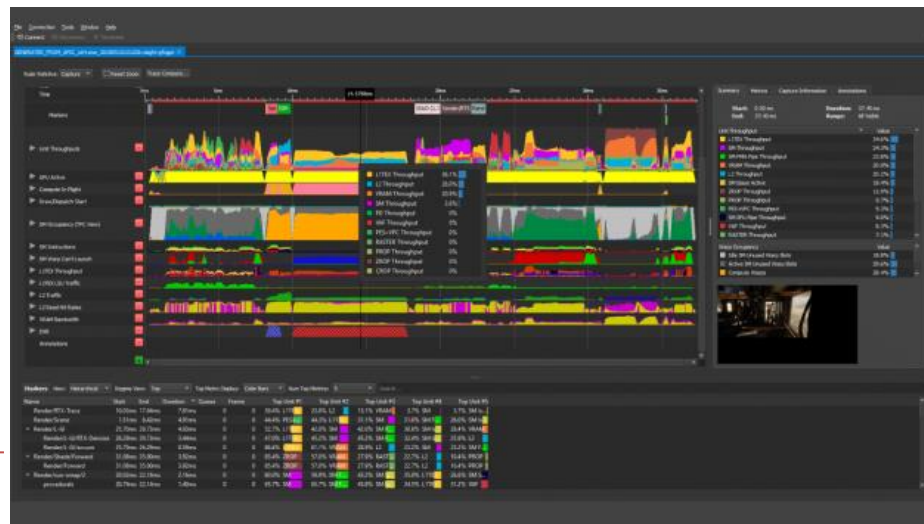
MobileNet:
Computing: 0.5 GFLOPS
Memory: 74MB
Computing intensity is 7

VGG-16:
Computing: 15 GFLOPS
Memory: 600MB
Computing intensity is 25

VGG fully utilizing the entire computational power of the 1080Ti

GPU Profiling Tools

- GPU profiling tools:
 - Nsight Systems: A system-wide profiler that visualizes CPU and GPU activity on a unified timeline to identify application-level bottlenecks.
 - Nsight Compute: A kernel-level tool providing deep-dive hardware metrics and performance tuning advice for individual GPU functions.
 - nvprof: A legacy command-line utility for basic timing and hardware counter collection in older CUDA environments.



Start to play with CUDA

test.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ +

⋮

🔍

<>

👁

🔑

📁

📅

Runtime

- Run all ⌘/Ctrl+F9
- Run before ⌘/Ctrl+F8
- Run the focused cell ⌘/Ctrl+Enter
- Run selection ⌘/Ctrl+Shift+Enter
- Run cell and below ⌘/Ctrl+F10
- Interrupt execution ⌘/Ctrl+M I
- Restart session ⌘/Ctrl+M .
- Restart session and run all
- Disconnect and delete runtime
- Change runtime type**
- Manage sessions
- View resources
- View runtime logs
- Deploy to Google Cloud Run

<https://colab.research.google.com/>

Change runtime type

Runtime type

Python 3 ▾

Hardware accelerator ?

☐ CPU ☒ T4 GPU ☐ H100 GPU ☐ A100 GPU

☐ L4 GPU ☐ v5e-1 TPU ☐ v6e-1 TPU

Want access to premium GPUs? [Purchase additional compute units](#)

Runtime version ?

Latest (recommended) ▾

Cancel Save

Start to play with CUDA

- Run the following command in the command line or terminal:

nvidia-smi

```
[4] ✓ Os !nvidia-smi
```

Mon Jan 5 20:24:05 2026

The version number of the NVIDIA System Management Interface (SMI) tool currently in use.

NVIDIA-SMI 550.54.15			Driver Version: 550.54.15			CUDA Version: 12.4		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	Tesla T4		Off	00000000:00:04.0	Off		0	
N/A	36C	P8	11W / 70W	0MiB / 15360MiB		0%	Default	N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
ID	ID	ID				Usage	
No running processes found							

Start to play with CUDA

- Run the following command in the command line or terminal:

nvidia-smi

```
[4] ✓ Os !nvidia-smi
```

Mon Jan 5 20:24:05 2026

The version number of the NVIDIA GPU driver installed on the system.

NVIDIA-SMI 550.54.15			Driver Version: 550.54.15			CUDA Version: 12.4		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	Tesla T4		Off	00000000:00:04.0	Off		0	
N/A	36C	P8	11W / 70W	0MiB / 15360MiB		0%	Default	N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
ID	ID	ID				Usage	
No running processes found							

Start to play with CUDA

- Run the following command in the command line or terminal:

nvidia-smi

```
[4] !nvidia-smi
✓ Os
Mon Jan 5 20:24:05 2026
```

NVIDIA-SMI 550.54.15		Driver Version: 550.54.15		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile
Fan	Temp	Pwr:Usage/Cap		Memory-Usage	GPU-Util
Perf					Uncorr. ECC
					Compute M.
					MIG M.
0	Tesla T4	Off	00000000:00:04.0	Off	0
N/A	36C	11W / 70W	0MiB / 15360MiB		Default
P8					N/A

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID				
No running processes found						

The CUDA version compatible with your driver.

This version information is important for ensuring software and hardware compatibility, troubleshooting problems, and determining whether software or driver updates are needed.

Start to play with CUDA

- Run the following command in the command line or terminal:

nvidia-smi

```
[4] ✓ Os !nvidia-smi
```

Mon Jan 5 20:24:05 2026

More info: GPU name/Temperature/Memory usage/GPU utilization...

NVIDIA-SMI 550.54.15			Driver Version: 550.54.15			CUDA Version: 12.4		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	Tesla T4		Off	00000000:00:04.0	Off		0	
N/A	36C	P8	11W / 70W	0MiB / 15360MiB		0%	Default	
							N/A	

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
ID	ID	ID				Usage	
No running processes found							

Write a helloworld using CUDA

```
!pip install nvcc4jupyter
%load_ext nvcc4jupyter
```

```
... Requirement already satisfied: nvcc4jupyter in /usr/local/lib/python3.12/dist-packages (1.2.1)
The nvcc4jupyter extension is already loaded. To reload it, use:
%reload_ext nvcc4jupyter
```

```
%writefile my_program.cu
#include <stdio.h>

// CUDA kernel
__global__ void hello_cuda() {
    int i = threadIdx.x;
    if (i < 5) {
        printf("Hello World CUDA from thread %d\n", i);
    }
}

int main() {
    // Launch 1 block with 5 threads
    printf("This is the main thread\n");
    hello_cuda<<<1, 5>>>();

    // Wait for GPU to finish
    cudaDeviceSynchronize();

    return 0;
}
```

```
... Overwriting my_program.cu
```

Setting up your Colab environment to compile and run CUDA C/C++ code directly within the notebook

<https://github.com/kevinsuo/CS4522/blob/main/helloworld.cu>

Hello world program: create 5 threads to print out hello world

Write a helloworld using CUDA

```
[70] !nvcc -gencode arch=compute_75,code=sm_75 -o my_program my_program.cu
```

```
[71] !./my_program
```

```
✓ 0s  
This is the main thread  
Hello World CUDA from thread 0  
Hello World CUDA from thread 1  
Hello World CUDA from thread 2  
Hello World CUDA from thread 3  
Hello World CUDA from thread 4
```

Compile the code

Run the hello world program

- **-gencode**: This is an `nvcc` (NVIDIA CUDA Compiler) option that allows you to specify the target architecture for which the CUDA code should be compiled. You can use it multiple times to compile for different architectures.
- **arch=compute_75**: This part specifies the **virtual architecture** (also known as the PTX virtual assembly target). `compute_75` refers to the generic compute capability version 7.5. When you compile with `arch=compute_XX`, `nvcc` generates **PTX (Parallel Thread Execution)** code. PTX is an intermediate, architecture-independent assembly language that can be JIT (Just-In-Time) compiled by the CUDA driver for any compatible physical GPU. This makes your compiled code more portable.
- **code=sm_75**: This part specifies the **real architecture** (also known as the SASS physical assembly target). `sm_75` refers to the Streaming Multiprocessor (SM) architecture version 7.5. When you compile with `code=sm_XX`, `nvcc` generates device-specific binary code (SASS - Streaming Assembler) directly for that specific physical architecture. This results in highly optimized code for the target GPU, often performing better than JIT-compiled PTX, but it is less portable.

Another example: Vector add

- Adds two vectors A and B of length 50,000
- Uses a CUDA kernel to compute the addition in parallel on the GPU
- Copies the result back to the CPU
- Verifies correctness by comparing GPU results with CPU computation



Another example: Vector add

- Adds two vectors A and B of length 50,000
- Uses a CUDA kernel to compute the addition in parallel on the GPU
- Copies the result back to the CPU
- Verifies correctness by comparing GPU results with CPU computation

https://github.com/kevinsuo/CS4522/blob/main/vector_add.cu



CUDA's memory management

- How to use CUDA's memory management functions:
 - Allocation: `cudaMalloc()`.
 - Copying: `cudaMemcpy()`, which can copy from host to device (H2D) or from device to host (D2H).
 - Deallocation: `cudaFree()`.

```
float *d_A;
```

```
cudaMalloc((void**)&d_A, sizeof(float) * N);  
// Allocate memory for N floats
```

```
cudaMemcpy(d_A, h_A, sizeof(float) * N, cudaMemcpyHostToDevice);  
// Copy from host to device
```

```
cudaFree(d_A);  
// Free the memory
```



CUDA's memory management

```
// Allocate input vectors A, B, and output vector C on the device
float* d_A = nullptr;
float* d_B = nullptr;
float* d_C = nullptr;
cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);

// Copy vector data from host to device
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

// Call the kernel function
int threadsPerBlock = 256;
int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, numElements);

// Copy the results from device back to host
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// Verify the results
for (int i = 0; i < numElements; ++i)
{
    if (fabs(h_A[i] + h_B[i] - h_C[i]) > 1e-5)
    {
        fprintf(stderr, "Result verification failed at element %d!\n", i);
        exit(EXIT_FAILURE);
    }
}

printf("Test PASSED\n");

// Free device memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

CUDA optimization

- **Using shared memory:** Shared memory is fast memory accessible by threads within each block. We can use it to store elements of A and B to reduce the number of global memory accesses.
- **Increasing thread parallelism:** We can try using more threads and blocks to perform the task, while also ensuring that the number of threads matches the hardware resources.
- **Improving memory access patterns:** We can ensure that memory accesses are coalesced, which reduces memory bandwidth usage.
- **Using instruction-level parallelism:** We can try reorganizing certain computation instructions to improve pipeline efficiency.



CUDA optimization

original

```
// kernel function
__global__ void vectorAdd(const float* A, const float* B, float* C, int numElements)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    if (i < numElements)
    {
        C[i] = A[i] + B[i];
    }
}
```

optimized

```
// Define thread block size and shared memory size
#define THREADS_PER_BLOCK 256
#define SHARED_MEM_SIZE THREADS_PER_BLOCK

// Kernel function definition
__global__ void vectorAdd(const float* A, const float* B, float* C, int numElements) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    // Use shared memory
    __shared__ float shared_A[SHARED_MEM_SIZE];
    __shared__ float shared_B[SHARED_MEM_SIZE];

    if (i < numElements) {
        shared_A[threadIdx.x] = A[i];
        shared_B[threadIdx.x] = B[i];
        __syncthreads(); // Ensure all threads have loaded data into shared memory

        C[i] = shared_A[threadIdx.x] + shared_B[threadIdx.x];
    }
}
```

https://github.com/kevinsuo/CS4522/blob/main/vector_add2.cu

Summary

- GPU Motivation & Background
- GPU Architecture Fundamentals
- GPU Programming Model

